

Projet d'Algorithmique Avancée

Sujet 8 : Table de Hachage

Langage de programmation : C

Nombre d'étudiants : 6

Classe : L1 B

Membres du groupe :

- Toky (327/LA/25-26)
- Mika (353/LA/25-26)
- Carmelah (317/LA/25-26)
- Iraina (359/LA/25-26)
- Hasina (363/LA/25-26)
- Océane (319/LA/25-26)

Présentation du projet

Le projet consiste à implémenter une table de hachage (hash table) en langage C. L'objectif est de reproduire les mécanismes internes d'une structure telle que HashMap en Java : calcul d'un indice à partir d'une fonction de hachage, gestion des collisions par chaînage, et redimensionnement automatique de la table lorsque le facteur de charge devient trop élevé.

Fonctionnalités développées :

- Fonction de hachage inspirée de String.hashCode() (Java) : $\text{hash} = 31 \times \text{hash} + \text{caractère}$
- Insertion (put) avec mise à jour si la clé existe déjà
- Recherche (get) d'une valeur à partir d'une clé
- Suppression (remove) d'un élément
- Gestion des collisions par chaînage (liste chaînée par bucket)
- Redimensionnement automatique (resize) et recalcul des indices (rehash)
- Jeux de tests et démonstration en ligne de commande

Répartition des tâches et explication du code

Toky : structures de base et fonction de hachage

Définition des deux structures principales du programme : Noeud, qui représente un élément de la liste chaînée (une clé, une valeur, et un pointeur vers le noeud suivant), et TableHashage, qui regroupe le tableau de buckets, la capacité actuelle et le nombre d'éléments stockés. Implémentation de la fonction hash_chaine(), qui calcule un nombre à partir d'une chaîne de caractères en reproduisant la formule utilisée par Java ($\text{hash} = 31 \times \text{hash} + \text{caractère}$), appliquée lettre par lettre dans une boucle. Implémentation également de etaler_bits(), qui mélange les bits de poids fort et de poids faible du hash ($h \text{ XOR } (h \text{ décalé de } 16 \text{ bits})$) afin de mieux répartir les clés entre les buckets, et de indice_bucket(), qui calcule l'indice final avec un ET binaire ($\text{hash AND } (\text{capacité} - 1)$), possible car la capacité est toujours une puissance de deux.

Mika : insertion et gestion des collisions (put)

Implémentation de la fonction `table_hashage_put()`. Cette fonction vérifie d'abord si le facteur de charge (nombre d'éléments divisé par la capacité) dépasse 0.75 ; si c'est le cas, elle déclenche un redimensionnement avant d'insérer le nouvel élément. Elle calcule ensuite l'indice du bucket correspondant à la clé, puis parcourt la liste chaînée de ce bucket pour vérifier si la clé existe déjà (comparaison avec `strcmp`). Si la clé est trouvée, sa valeur est simplement mise à jour. Sinon, un nouveau noeud est créé (avec allocation dynamique de la clé par `malloc` et `strcpy`) et inséré en tête de la liste chaînée du bucket concerné.

Iraina : recherche et suppression (get, remove)

Implémentation des fonctions `table_hashage_get()` et `table_hashage_remove()`. La fonction `get()` recalcule l'indice du bucket à partir de la clé, puis parcourt la liste chaînée en comparant chaque clé avec `strcmp` jusqu'à trouver une correspondance ; si aucune clé ne correspond, la fonction retourne 0. La fonction `remove()` suit un principe similaire, mais garde en mémoire le noeud précédent pendant le parcours afin de pouvoir retirer le noeud trouvé de la liste chaînée (en reliant le précédent directement au suivant), puis libère la mémoire allouée pour ce noeud et sa clé.

Hasina : redimensionnement automatique (resize)

Implémentation de la fonction `redimensionner()`, appelée automatiquement par `put()` lorsque le facteur de charge devient trop élevé. Cette fonction double la capacité du tableau de buckets, puis reprend chaque élément de l'ancien tableau un par un pour recalculer son indice dans le nouveau tableau (car l'indice dépend directement de la capacité, via `hash AND (capacite - 1)`). Chaque noeud est ainsi replacé dans le bon bucket du nouveau tableau, sans recréer les noeuds : seuls les pointeurs sont réorganisés, ce qui évite des allocations mémoire inutiles.

Océane : jeux de tests et vérification mémoire

Conception des scénarios de test exécutés dans la fonction `main()`. Ces tests couvrent l'insertion de plusieurs clés (dont deux qui provoquent volontairement une collision sur le même bucket, afin de vérifier le chaînage), la mise à jour d'une clé existante, la suppression d'un élément, ainsi que l'ajout de plusieurs clés supplémentaires pour déclencher le redimensionnement automatique et vérifier que les éléments sont correctement replacés après le rehash. Vérification également que chaque allocation mémoire (`malloc`, `calloc`) est bien libérée par les fonctions `liberer_liste()` et `liberer_table_hashage()`, afin d'éviter toute fuite mémoire.

Carmelah : documentation et relecture finale

Rédaction des commentaires du code afin d'expliquer le rôle de chaque fonction et de chaque étape importante (calcul du hash, gestion des collisions, redimensionnement). Relecture de l'ensemble du programme pour vérifier la cohérence des noms de fonctions et de variables, s'assurer que le code compile sans avertissement (option `-Wall`), et confirmer que le comportement du programme correspond bien à ce qui est décrit dans ce rapport.

Complexité algorithmique

`hash_chaine(cle)` : la fonction parcourt chaque caractère de la clé une seule fois, sa complexité est donc $O(k)$, où k est la longueur de la clé.

`table_hashage_put()`, `table_hashage_get()`, `table_hashage_remove()` : en moyenne, lorsque les clés sont bien réparties entre les buckets, ces trois opérations s'effectuent en temps constant, soit $O(1)$, car la liste chaînée de chaque bucket reste très courte. Dans le pire des cas, si un grand nombre de clés se retrouvent dans le même bucket (mauvaise répartition du hachage), la liste chaînée de ce bucket peut contenir jusqu'à n éléments, et la complexité devient alors $O(n)$.

`redimensionner()` : cette fonction doit parcourir et remplacer tous les éléments existants dans le nouveau tableau, sa complexité est donc $O(n)$. Cependant, comme elle ne se déclenche qu'occasionnellement (lorsque le facteur de charge dépasse 0.75), son coût est amorti sur l'ensemble des insertions : en moyenne, chaque insertion reste $O(1)$ amorti.

En résumé, la table de hachage offre des opérations d'insertion, de recherche et de suppression en $O(1)$ en moyenne, ce qui est l'intérêt principal de cette structure de données par rapport à une simple liste chaînée ($O(n)$) ou un tableau trié ($O(\log n)$ pour la recherche par dichotomie).